Ton Duc Thang University
Faculty of Information Technology

FINAL PROJECT

Course: Introduction to Artificial Intelligence

Duration: 06 weeks

I. Formation

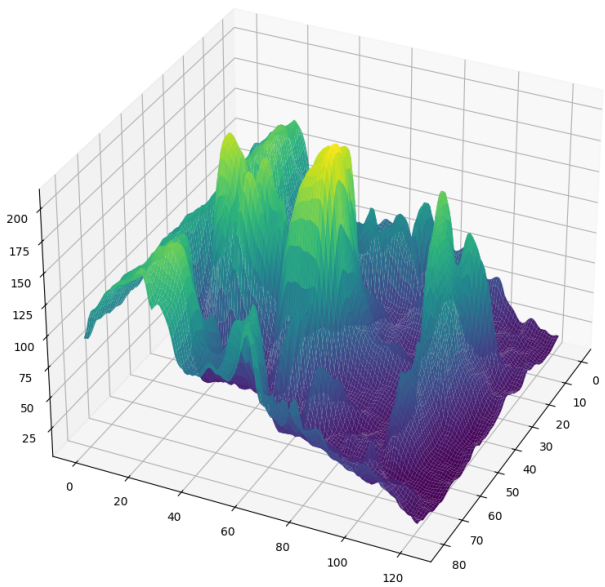
1. The project is conducted in groups of students.
2. Student groups conduct required tasks and submit the project following instructions below.

II. Tasks

a) Task 1 (2.0 point(s)): Local Search

Descriptions:

- Given the project folder **LocalSearch** consisting of
  - **monalisa.jpg**: an image for generating the state space.
  - **viz3d.py**: a source code file with examples for loading state spaces, visualizing spaces, and drawing moving paths using the matplotlib library.
- In this problem, each state of the space contains two non-negative integral attributes, x and y. Corresponding to each state (x, y) is an evaluation function value, z ∈ [0, 255]. The larger the value of z, the better the state. Visualizing all sets of numbers (x, y, z), we have a curve as shown below.



Requirements:

1. *(0.5 point(s))*: Formulate the search problem and implement a corresponding class in **problem.py** with reasonable attributes and methods. There must be a method for illustrating the surface and one for drawing search paths.
2. *(1.0 point(s))*: Implement the Simulated Annealing Search (SAS) algorithm to search for the location (x, y) with the largest z-value, starting from O (0, 0) in **search.py**,
   - *(0.25 point(s))*: For each state, there are at most 8 neighbors according to 8 directions {N, E, W, S, NW, SW, NE, SE}. States at borders miss several neighbors.

NW	N	NE
W		E
SW	S	SE

   - *(0.25 point(s))*: Propose a ***schedule*** function (***sd1***) as a mapping between time step *t* and the energy *T*, in which *t* increases then *T* decreases.
   - *(0.25 point(s))*: Propose a ***schedule*** function (***sd2***) to receive additional energy at each state, where the additional energy is inversely proportional to the slope of the surface.
   - *(0.25 point(s))*: Propose an experiment to compare the best z-values found using ***sd1*** versus ***sd2***, and their running time (the number of time steps).
3. *(0.5 point(s)):* Organize the program regarding to the OOP model, ensure source code is compact and reasonable.

Notice:

- Recommended editor: Visual Studio Code

b) Task 2 (2.0 point(s)): Adversarial Search

Descriptions:

- Students implement the Tic-Tac-Toe game with the size of 3 x 3, in which the user plays against the computer.

- For each player, there can be at most 4 symbols (X or O) in the board. If the player adds the 5th symbol to the board, then the oldest symbol is removed automatically.

Requirements:

1. *(0.5 point(s)):* Implement the game with friendly UI in which the human user can play against the computer using mouse clicks (Hint: use **PyGame** library).
2. *(1.0 (point(s)):* Implement Alpha-Beta Search algorithm to control the computer.
    - *(0.5 point(s)):* Propose utility values for terminal states to implement the algorithm.
    - *(0.5 point(s)):* Symbols are removed automatically when the 5th one of the same kinds is added, then the search tree may be infinite. Students proposed an approach to deal with the infinite search tree and implement the approach to compare the performance to the baseline algorithm.
3. *(0.5 point(s))*: Organize the program regarding to the OOP model, ensure source code is compact and reasonable.

Notice:

- Recommended editor: Visual Studio Code.

c) Task 3 (2.0 point(s)): Constraint Satisfaction Problems

Descriptions:

- Given a *m x n* matrix, each cell consists of a non-negative integer or it is blank.
- Each cell has 9 "adjacent" neighbors, including itself and 8 cells around.
- The player colors cells by red or green so that the number of green cells which are "adjacent" to a cell matches the number inside.

- There is no constraint for blank cells.

Input data file (left) – Result (right)

Requirements:

1. *(0.5 point(s)):* Determine all possible constraints in the given CSP, give examples for each.
2. *(1.0 point(s)):* Implement an agent taking in a matrix and returning a solution.
   - *(0.25 point(s))* Generate all possible constraints and convert them into Conjunctive Normal Form (CNF) clauses.
   - *(0.25 point(s))* Discover general rules to generate CNF clauses without enumerating all possible ones in order to reduce the number of clauses. Note that no duplicated clauses are allowed.
   - *(0.25 point(s))* Find a model satisfying all clauses using **Glucose3 of PySAT**.
   - *(0.25 point(s))* Implement a visualization functionality that helps the user to evaluate the agent's result.
3. *(0.5 point(s):* Organize the program regarding to the OOP model, ensure source code is compact and reasonable.

Notice:

- Recommended editor: Visual Studio Code.

d) Task 4 (2.0 point(s)): Propositional Logic Inference

Descriptions:

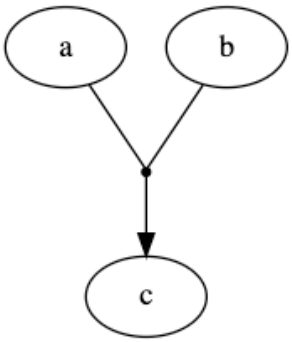
- Given a data file consisting of definite clauses, one clause per row, for example,

```
-p q
-l -m p
-b -l m
-a -p l
-a -b l
A
b
```

  - Literals are separated by spaces
  - "-" means negation
- Students implement a class named **Clause** to represent clauses with related information such as the premise, the conclusion, the number of known literals of the premise, etc.

Requirements:

1. *(0.5 (points)):* Draw a direct graph, using **`graphviz`**, to illustrate the given knowledge base read from the given data file. An implication can be displayed as below. Besides, organize the program regarding to the OOP model, ensure source code is compact and reasonable.



2. *(0.5 (points)):* Implement the Forward Chaining algorithm to verify whether the given knowledge base entails a symbol `q`, input by the user. Students refer to the given pseudo code in the lecture.
3. *(0.5 (points)):* Implement the Backward Chaining algorithm to verify whether the given knowledge base entails a symbol *q*, input by the user.

4. *(0.5 (points)):* Propose an experiment to contrast the performance of Forward versus Backward Chaining algorithm. Targeted criteria include the number of "inferred" nodes in the graph.

Notice:

- Recommended editor: IPython Notebook.

e) Task 5 (2.0 point(s)): Report and Interview

- Student groups compose the project report using the IEEE conference proceeding template.
- Recommended editor: Overleaf.
- Selective contents:
  - *Title*: the project title
  - *Authors*: group member's information, the lecturer is appended as the last author.
  - *Abstract*: summarize the project requirements, approaches, experimental results, and levels of completion.
  - Each following section presents a task in the project, with a meaningful and human-readable title. Briefly introduce the approach to tackle the problem and illustrate results with related figures/tables, etc.
  - "*Contributions*" section: individual tasks, individual completion levels (0%-100%).
  - "*Self-evaluation*" section: self-evaluate task completion and estimate scores.
  - "*Conclusion*" section: summarize the project requirements, approaches, experimental results, and levels of completion.
- References are in the IEEE format.
- Maximal length is **05** pages.
- Students must take an interview to justify your work in the project. If you can not explain clearly and exactly ideas nor implementations, then you get 0.0 points for related tasks.

III. Submission

- Create a folder whose name is in the format

  final_<Project Group ID>_<Student ID>

  - **source:** source code files
  - **report.pdf:** report of the project
  - **demo.txt**: a URL to the demo video.
- Students maintain outputs of all cells in ipynb files.
- Compress the folder into a zip file and submit by the deadline.
- **Every team member must submit the project individually**.

IV. Policy

- **Student groups submitting late get 0.0 points for each member.**
- **Missing required materials in the submission loses at least 50% points of the presentation.**
- **Copying source code on the internet/other students, sharing your work with other groups, etc. cause 0.0 points for all related groups.**
- **If there exist any signs of illegal copying or sharing of the assignment, then extra interviews are conducted to verify student groups' work.**
- **AI tools are forbidden in this project. If any abnormal symbols generated by AI tools appear in the source code files, the group will receive a score of 0.0 for the corresponding task.**

-- THE END --